F1

Knowledge Graph Explorer

Web Semantics — Practical Assignment Report

Author	Rodrigo Abreu (113626), Hugo Ribeiro ()
Course	Web Semantics
Year	2025–2026
Technology	Django 4 · GraphDB · SPARQL · RDF

April 12, 2026

Contents

1. Introduction

Formula 1 is the premier class of single-seater auto racing sanctioned by the Fédération Internationale de l'Automobile (FIA). Since its inaugural season in 1950, it has grown into one of the most data-rich sports in the world, with precise records of every driver, constructor, race result, qualifying session, lap time, and pit stop stored in structured datasets.

This project constructs a **semantic web application** around Formula 1 historical data covering the 1950–2024 seasons. The data is modelled as an RDF knowledge graph stored in **GraphDB** (a native RDF triple store with RDFS+ inference), exposed through a **SPARQL** query interface, and presented to users via a **Django** web application styled with the visual identity of the sport.

1.1. Objectives

- Transform flat CSV data from the Formula 1 World Championship (1950 - 2024) dataset into a well-structured RDF knowledge graph using custom Python scripts.
- Model the domain with a consistent namespace and predicate vocabulary that enables efficient SPARQL queries over millions of triples.
- Build a full-featured web UI allowing public users to explore drivers, constructors, circuits, races, and seasons in detail.
- Provide an authenticated admin panel allowing staff to create, update, and delete core entities directly in the knowledge graph via SPARQL UPDATE, including relation-backed fields.
- Add higher-level knowledge-graph operations such as batch CSV import, dry-run preview, rollback, and natural-language querying over SPARQL.

1.2. Technology Stack

Layer	Technology	Role
Knowledge Store	GraphDB 10 (Ontotext)	RDF triple store, SPARQL endpoint
Query Language	SPARQL 1.1	SELECT, UPDATE (INSERT/DELETE)
Data Format	N-Triples (.nt)	RDF serialisation for bulk import
Web Framework	Django 4.x (Python)	Views, templates, URL routing
SPARQL Client	SPARQLWrapper	HTTP bridge to GraphDB endpoint
Serialisation	RDFLib	CSV → RDF conversion
Auth	Django contrib.auth	Session-based login for admin
Frontend	Vanilla HTML/CSS/JS	Responsive dark-theme UI
LLM Integration	Gemini API	Natural-language to SPARQL assistant

2. Data, Sources, and Transformation Process

2.1. Data Source

The raw data originates from the **Formula 1 World Championship (1950 - 2024)** (https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020?select=circuits.csv), a free API and database export providing structured Formula 1 statistics from 1950 to the present. For this project the data was downloaded as a CSV export and covers the **1950–2024** seasons.

The dataset consists of **13 CSV files**, each representing a distinct entity type or relationship:

CSV File	RDF Class	Key Fields
`seasons.csv`	Season	year, url
`circuits.csv`	Circuit	circuitId, name, location, country, lat, lng, alt, url
`constructors.csv`	Constructor	constructorId, name, nationality, url
`drivers.csv`	Driver	driverId, forename, surname, code, number, dob, nationality, url
`races.csv`	Race	raceId, year, round, name, date, circuitId (FK), url
`results.csv`	Result	resultId, raceId, driverId, constructorId, grid, positionOrder, points, laps, time, statusId, fastestLapTime, fastestLapSpeed
`qualifying.csv`	QualifyingResult	qualifyId, raceId, driverId, constructorId, position, q1, q2, q3
`pit_stops.csv`	PitStop	raceId, driverId, stop, lap, milliseconds, duration
`lap_times.csv`	LapTime	raceId, driverId, lap, position, time, milliseconds
`driver_standings.csv`	DriverStanding	driverStandingsId, raceId, driverId, points, position, wins
`constructor_standings.csv`	ConstructorStanding	constructorStandingsId, raceId, constructorId, points, position, wins
`constructor_results.csv`	ConstructorResult	constructorResultsId, raceId, constructorId, points, status
`status.csv`	Status	statusId, status (text label)

2.2. RDF Namespace and Vocabulary

A custom namespace `http://example.org/f1/` is used for all domain predicates, and `http://example.org/resource/` is used as the base for all entity URIs. Standard W3C vocabularies are used where appropriate.

```
PREFIX f1:   <http://example.org/f1/>
PREFIX res:  <http://example.org/resource/>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
```

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
```

Listing 1: SPARQL namespace prefixes used throughout the application

2.2.1. URI Construction

Entity URIs follow a consistent pattern: `res:{kind}/{id}`, where `kind` is a lowercase slug derived from the entity class and `id` is the numeric identifier from the source CSV.

Entity	Example URI
Driver	`res:driver/1`
Constructor	`res:constructor/6`
Circuit	`res:circuit/6`
Race	`res:race/841`
Result	`res:result/1`
QualifyingResult	`res:qualifying-result/1`
PitStop	`res:pit-stop/{raceId}/{driverId}/{stop}`
LapTime	`res:lap-time/{raceId}/{driverId}/{lap}`
Status	`res:status/1`

2.3. Transformation Script

The Python script `scripts/csv_to_rdf.py` performs the full CSV-to-RDF conversion using RDFLib. The process for each CSV file is:

1. Read each row with `csv.DictReader`.
2. Construct the subject URI from the configured `id_columns`.
3. For each column, cast the value to the appropriate XSD datatype (`xsd:integer`, `xsd:decimal`, `xsd:date`, `xsd:gYear`) or leave it as a plain literal.
4. For foreign key columns (e.g. `raceId`, `driverId`), emit an object property triple linking to the target entity URI.
5. Emit a `rdfs:label` triple with a human-readable label derived from available name fields.
6. Serialise the complete graph as **N-Triples** (`.nt`) for efficient bulk import into GraphDB.

2.3.1. Data Type Mapping

Column group	XSD type	Example columns
Numeric identifiers/counts	`xsd:integer`	driverId, raceId, laps, position
Floating-point values	`xsd:decimal`	lat, lng, points, fastestLapSpeed
Calendar dates	`xsd:date`	dob, date, fp1_date
Calendar years	`xsd:gYear`	year
Time strings	plain literal	time, fastestLapTime, q1, q2, q3
External URLs	`rdfs:seeAlso`	url

2.3.2. Derived Labels

Because SPARQL queries use `rdfs:label` for display, the script generates a label for every entity at conversion time. For drivers, the label is the concatenation of `forename` and `surname`. For races it uses the `name` column. For seasons the label is "Season YYYY".

2.3.3. Sprint Results Exclusion

The original dataset includes a `sprint_results.csv` file recording results from Saturday sprint races. Sprint results use the same `resultId` sequence as regular race results and link to the same race entities via `f1:race`. This caused **duplicate rows** when querying race results: for any sprint weekend, both a `res:result/N` and a `res:sprint-result/N` entity would match the `f1:resultId` anchor, effectively doubling every driver's entry in the results table.

Sprint results were therefore excluded from the conversion script. A SPARQL `FILTER` guard is also applied in the race detail query as a defensive measure for any existing data in the store:

```
FILTER(!CONTAINS(STR(?res), "/sprint-result/"))
```

Listing 2: Filter excluding sprint-result URIs from race result queries

2.4. Loading into GraphDB

The generated `.nt` file is loaded into GraphDB using the **Import → Upload RDF files** interface in the GraphDB Workbench, targeting the named graph `http://example.org/graph/formula1`. The repository is configured with **RDFS+ (Optimized)** inference enabled, which allows entailments over `rdfs:subClassOf` and `rdfs:subPropertyOf` chains.

3. Operations on the Data (SPARQL Queries)

All SPARQL queries are executed via the `GraphDBClient` class in `championship/services/graphdb.py`, which prepends the five namespace prefixes to every query. The client exposes two methods:

- `query(sparql)` — executes a SELECT and returns a list of dictionaries mapping variable names to string values.
- `run_update(sparql)` — executes a SPARQL UPDATE (INSERT or DELETE) against the `/statements` endpoint.

3.1. Anchoring Strategy

A central design decision is the use of **property anchors** rather than `rdf:type` assertions for entity discovery. GraphDB's RDFS+ inference engine processes `rdf:type` through the class hierarchy, which is expensive on large graphs. Instead, every query starts with a property that is unique to the target entity class:

Entity	Anchor predicate
Driver	`f1:driverId`
Constructor	`f1:constructorId`
Circuit	`f1:circuitRef` (not `f1:circuitId` — races also carry this)
Race	`f1:round`
Result	`f1:resultId`
QualifyingResult	`f1:qualifyId`
PitStop	`f1:stop`
LapTime	`f1:position` (lap_times only; pit_stops do not have this)
DriverStanding	`f1:driverStandingsId`
ConstructorStanding	`f1:constructorStandingsId`

3.2. Year Filtering

Race years are stored as `xsd:gYear`. Direct numeric comparison (`FILTER(?year = 2024)`) fails because the literal type does not match `xsd:integer`. All season-scoped queries therefore coerce to string:

```
FILTER(STR(?yr) = "2024")
```

3.3. Read Queries (SELECT)

3.3.1. Homepage Statistics

The home page displays live aggregate counts queried directly from the graph, ensuring they always reflect the current state of the knowledge base.

```
SELECT
  (COUNT(DISTINCT ?season)      AS ?seasons)
  (COUNT(DISTINCT ?circuit)     AS ?circuits)
  (COUNT(DISTINCT ?driver)      AS ?drivers)
  (COUNT(DISTINCT ?constructor) AS ?constructors)
  (COUNT(DISTINCT ?race)        AS ?races)
WHERE {
  ?race      f1:round       ?round ;
             f1:year        ?season ;
             f1:circuit     ?circuit .
  ?res       f1:resultId    ?anyId ;
             f1:race        ?race ;
             f1:driver      ?driver ;
             f1:constructor ?constructor .
}
```

Listing 3: Aggregate entity counts for the homepage

3.3.2. Driver List with Career Statistics

Three queries are executed in sequence to build the drivers list page: one for basic fields, one for career year ranges, and one for podium breakdowns.

```
SELECT ?driver
       (COUNT(*) AS ?races)
```

```
       (MIN(STR(?yr)) AS ?firstYear)
       (MAX(STR(?yr)) AS ?lastYear)
WHERE {
  ?r f1:resultId ?anyId ;
     f1:driver   ?driver ;
     f1:race     ?race .
  ?race f1:year ?yr .
}
GROUP BY ?driver
```

Listing 4: Career year range per driver (MIN/MAX over race years)

```
SELECT ?driver ?pos (COUNT(*) AS ?cnt) WHERE {
  ?r f1:resultId    ?anyId ;
     f1:driver      ?driver ;
     f1:positionOrder ?pos .
  FILTER(?pos IN (1, 2, 3))
}
GROUP BY ?driver ?pos
```

Listing 5: Podium breakdown (positions 1, 2, 3) per driver

3.3.3. Constructor Detail — Driver Roster

This query uses conditional aggregation (`SUM(IF(...))`) to compute per-driver wins and podiums without leaving the SPARQL layer.

```
SELECT ?driver ?driverLabel
       (COUNT(*) AS ?races)
       (SUM(IF(?pos = 1, 1, 0))          AS ?wins)
       (SUM(IF(?pos IN (1,2,3), 1, 0))   AS ?podiums)
       (MIN(?year) AS ?firstYear)
       (MAX(?year) AS ?lastYear)
WHERE {
  ?r f1:resultId    ?anyId ;
     f1:constructor <{constructorUri}> ;
     f1:driver      ?driver ;
     f1:positionOrder ?pos ;
     f1:race        ?race .
  ?race   f1:year    ?year .
  ?driver rdfs:label ?driverLabel .
}
GROUP BY ?driver ?driverLabel
ORDER BY DESC(?wins) DESC(?races)
```

Listing 6: Driver roster with win and podium counts for a constructor

3.3.4. Race Detail — Results with Finish Status

Results for a race are retrieved in a single query. The finish status (e.g. “Finished”, “Engine”, “Collision”) is joined via `f1:statusId` linking results to status entities. Both joins are wrapped in `OPTIONAL` so drivers without a recorded status still appear.

```
SELECT ?positionOrder ?grid ?laps ?time ?points ?statusText
       ?driverLabel ?driverId ?constructorLabel WHERE {
```

```
  ?res f1:resultId     ?anyId ;
       f1:race         <{raceUri}> ;
       f1:positionOrder ?positionOrder ;
       f1:driver        ?driver ;
       f1:constructor   ?constructor .
  FILTER(!CONTAINS(STR(?res), "/sprint-result/"))
  OPTIONAL { ?res f1:grid   ?grid   }
  OPTIONAL { ?res f1:laps   ?laps   }
  OPTIONAL { ?res f1:time   ?time   }
  OPTIONAL { ?res f1:points ?points }
  OPTIONAL {
    ?res       f1:statusId ?statusId .
    ?statusEnt f1:statusId ?statusId ;
               f1:status   ?statusText .
    FILTER(!CONTAINS(STR(?statusEnt), "/result/"))
  }
  ?driver      rdfs:label ?driverLabel ;
               f1:driverId ?driverId .
  ?constructor rdfs:label ?constructorLabel .
}
ORDER BY ?positionOrder
```

Listing 7: Race results with finish status join

3.3.5. Pit Stop Aggregation

Pit stop data is aggregated per driver per race. The `f1:stop` property serves as the anchor (unique to pit stop entities). The query computes total pit time, fastest single stop time, and the lap on which the fastest stop occurred.

```
SELECT ?driverLabel ?driverId
       (COUNT(*)          AS ?stops)
       (SUM(?ms)          AS ?totalMs)
       (MIN(?ms)          AS ?fastestMs)
       (SAMPLE(?fastestLap) AS ?fastestLap)
WHERE {
  ?ps f1:stop         ?stop ;
      f1:race         <{raceUri}> ;
      f1:driver       ?driver ;
      f1:milliseconds ?ms ;
      f1:lap          ?fastestLap .
  ?driver rdfs:label ?driverLabel ;
          f1:driverId ?driverId .
}
GROUP BY ?driverLabel ?driverId
ORDER BY ?driverLabel
```

Listing 8: Per-driver pit stop aggregation for a race

3.3.6. Season Detail — Championship Standings

Standings are stored per-race (cumulative totals after each round). To obtain *final* standings for a season, the query orders by round descending and the application keeps only the first occurrence per driver/constructor.

```
SELECT ?driverLabel ?driverId
```

```
        ?points ?position ?wins ?round WHERE {
  ?ds f1:driverStandingsId ?anyId ;
      f1:race     ?race ;
      f1:driver   ?driver ;
      f1:points   ?points ;
      f1:position ?position ;
      f1:wins     ?wins .
  ?race f1:year  ?yr ;
        f1:round ?round .
  FILTER(STR(?yr) = "{year}")
  ?driver rdfs:label ?driverLabel ;
          f1:driverId ?driverId .
}
ORDER BY DESC(?round) ?position
```

Listing 9: Driver championship standings for a season (final round)

3.3.7. Circuit Fastest Lap

Lap time entities carry a `f1:position` property (finishing position within that lap). Position 1 identifies the fastest lap in each lap. However, since the circuit detail page seeks the all-time fastest lap recorded at that circuit across all races, the query retrieves the minimum milliseconds value.

```
SELECT ?driverLabel ?driverId ?time ?ms ?year WHERE {
  ?race f1:circuit <{circuitUri}> ;
        f1:year     ?year .
  ?lt   f1:race         ?race ;
        f1:driver       ?driver ;
        f1:milliseconds ?ms ;
        f1:time         ?time ;
        f1:lap          ?lap ;
        f1:position     ?pos .
  ?driver rdfs:label ?driverLabel ;
          f1:driverId ?driverId .
  FILTER(?ms > 60000)
}
ORDER BY ASC(?ms)
LIMIT 5
```

Listing 10: Fastest laps ever recorded at a circuit

3.4. Write Operations (SPARQL UPDATE)

The admin panel performs SPARQL UPDATE queries via the `/statements` endpoint. All mutations are expressed using standard SPARQL 1.1 UPDATE syntax.

3.4.1. Entity Creation (INSERT DATA)

New entities are created with a single `INSERT DATA` block. Before inserting, a helper query computes the next available ID by finding the current maximum for the target class:

```
SELECT (MAX(?id) AS ?maxId) WHERE {
  ?e f1:driverId ?id .
}
```

Listing 11: Maximum ID query for sequential ID generation (shown for drivers; analogous for all entity types)

The same pattern is applied for `f1:constructorId`, `f1:circuitId`, `f1:raceId`, etc. Each entity type has a well-defined set of required and optional predicates:

```
INSERT DATA {
  <http://example.org/resource/driver/862>
    rdfs:label      "George Russell" ;
    f1:driverId     862 ;
    f1:driverRef    "russell" ;
    f1:forename     "George" ;
    f1:surname      "Russell" ;
    f1:code         "RUS" ;
    f1:number       63 ;
    f1:dob          "1998-02-15"^^xsd:date ;
    f1:nationality "British" ;
    rdfs:seeAlso    <https://en.wikipedia.org/wiki/George_Russell> .
}
```

Listing 12: INSERT DATA for a new driver with all fields

```
INSERT DATA {
  <http://example.org/resource/constructor/212>
    rdfs:label         "Aston Martin" ;
    f1:constructorId   212 ;
    f1:constructorRef "aston_martin" ;
    f1:name            "Aston Martin" ;
    f1:nationality     "British" ;
    rdfs:seeAlso
       <https://en.wikipedia.org/wiki/Aston_Martin_in_Formula_One> .
}
```

Listing 13: INSERT DATA for a new constructor

```
INSERT DATA {
  <http://example.org/resource/circuit/78>
    rdfs:label     "Las Vegas Street Circuit" ;
    f1:circuitId   78 ;
    f1:circuitRef "las_vegas" ;
    f1:name        "Las Vegas Street Circuit" ;
    f1:location    "Las Vegas" ;
    f1:country     "USA" ;
    f1:lat         36.1699 ;
    f1:lng         -115.1398 ;
    rdfs:seeAlso
       <https://en.wikipedia.org/wiki/Las_Vegas_Grand_Prix> .
}
```

Listing 14: INSERT DATA for a new circuit

```
INSERT DATA {
  <http://example.org/resource/race/1126>
    rdfs:label   "Las Vegas Grand Prix" ;
    f1:raceId    1126 ;
```

```
    f1:name      "Las Vegas Grand Prix" ;
    f1:year      "2025"^^xsd:gYear ;
    f1:round     22 ;
    f1:circuit   <http://example.org/resource/circuit/78> ;
    f1:date      "2025-11-22"^^xsd:date ;
    rdfs:seeAlso
       <https://en.wikipedia.org/wiki/2025_Las_Vegas_Grand_Prix> .
}
```

Listing 15: INSERT DATA for a new race, linking to an existing circuit

```
INSERT DATA {
  <http://example.org/resource/season/2025>
    f1:seasonYear "2025"^^xsd:gYear ;
    rdfs:label    "Season 2025" ;
    rdfs:seeAlso
       <https://en.wikipedia.org/wiki/2025_Formula_One_World_Championship>
       .
}
```

Listing 16: INSERT DATA for a new season

3.4.2. Entity Update (DELETE + INSERT)

Updating an entity requires removing all its current triples first, then reinserting the complete set of new values. Both operations are sent in a single SPARQL UPDATE request, separated by a semicolon, which provides pseudo-atomic semantics:

```
DELETE WHERE {
  <http://example.org/resource/driver/862> ?p ?o
} ;
INSERT DATA {
  <http://example.org/resource/driver/862>
    rdfs:label     "George Russell" ;
    f1:driverId    862 ;
    f1:nationality "British" ;
    f1:number      63 ;
    ... .
}
```

Listing 17: DELETE + INSERT pattern for entity updates (identical structure for all entity types)

This pattern is identical for all five entity types (drivers, constructors, circuits, races, seasons). The only difference is the subject URI prefix and the set of predicates emitted. Because the DELETE removes *all* predicates before the INSERT, optional fields that were previously set and are now absent (e.g. a driver code that was cleared) are cleanly removed from the graph.

For relation-backed forms the same pattern is reused. For example, editing a driver can now also replace or clear the `f1:constructor` object property, allowing the admin panel to modify not only scalar literals but also links between RDF entities.

3.4.3. Entity Deletion (DELETE WHERE)

Deleting an entity removes all triples where the entity appears as the subject:

```
DELETE WHERE {
  <http://example.org/resource/driver/862> ?p ?o .
}
```

Listing 18: DELETE WHERE for entity removal

The same one-liner applies to constructors, circuits, races, and seasons, only the URI changes. The admin panel adds a confirmation modal before dispatching the DELETE to guard against accidental deletions.

> **Note on derived data:** Metrics such as race wins, podium counts, and championship points are *derived* by aggregating race result triples at query time. They are never stored directly and therefore do not need to be maintained through UPDATE operations. Deleting a driver removes only the driver entity; the result triples that reference it are preserved, though they become orphaned references.

3.4.4. Batch Import, Preview, and Rollback

The admin panel was extended with a race-results import workflow under `/admin-panel/imports/race-r` An administrator selects a race, uploads a CSV file, and receives a dry-run preview before any SPARQL UPDATE is sent to GraphDB. The preview shows:

- new result triples to be inserted;
- existing result triples that will be replaced;
- unresolved references (driver, constructor, or status IDs not found in the graph);
- duplicate or conflicting rows in the uploaded CSV.

Only after explicit confirmation is the generated SPARQL batch update executed. Each confirmed import stores a corresponding rollback record in Django's relational database, including enough metadata to reverse the exact SPARQL operation later. The admin dashboard exposes an "undo latest batch" style workflow for these recorded imports.

4. Application Functionalities (UI)

The application is built with Django, rendering server-side HTML templates and communicating with GraphDB over HTTP. There is no JavaScript framework; all interactivity is achieved with vanilla JS and CSS custom properties.

4.1. Public Interface

4.1.1. Homepage

The homepage presents an overview of the knowledge graph with five live stat tiles (seasons, circuits, drivers, constructors, races) fetched via a single aggregation query. Below the stats, five navigation cards link to the main entity sections. A connectivity indicator in

the footer shows whether the GraphDB endpoint is reachable. The page applies a dark motorsport aesthetic with the F1 red accent colour throughout.

4.1.2. Drivers Section

The drivers list page provides:

- **Podium display** — the top three all-time wins leaders shown as styled cards with photo slots for featured drivers.
- **Filterable table** — server-side filtering by name, nationality, and medal type (gold/silver/bronze podiums).
- **Sortable columns** — clicking any column header re-queries the server with the chosen sort dimension.
- **Cards / table toggle** — client-side view switch persisted in `localStorage`.
- **Career span** — each driver shows their first and last active season (e.g. "2007–2016").

The driver detail page shows biographical info, career statistics (races, wins, podiums, championships), a horizontal constructor timeline, top circuits, and a table of the driver's best results.

4.1.3. Constructors Section

The constructors list page mirrors the drivers page in structure, providing a top-three wins podium (with logo image slots for prominent teams), a nationality filter, server-side column sorting, and a card/table view toggle. Each card displays the team's nationality, estimated founding year, total race count, and win tally.

The constructor detail page is divided into four areas:

- **Hero section** — team name, nationality, and Wikipedia link.
- **Stat tiles** — total races entered, total wins, total podiums, and the number of distinct drivers who raced for the team.
- **Driver roster table** — every driver who competed for the constructor, with their first and last season, race count, wins, and podium count. Rows are sorted by wins descending. These figures are computed entirely in SPARQL using `SUM(IF(...))` conditional aggregation; no post-processing is required.
- **Top circuits** — the six circuits where the constructor has entered the most races, shown as labelled chips. Useful for identifying home-track advantages or high-frequency venues.

4.1.4. Circuits Section

The circuits list page features a podium of the three circuits that have hosted the most Grands Prix, each with an image slot for iconic venues (Monaco, Monza, Silverstone). Below the podium, a card/table grid lists all circuits with their country, location, and race count badge. Retired circuits are visually distinguished with a "Retired" status chip.

The circuit detail page provides:

- **Hero section** — circuit name, location, country, and geographic coordinates (latitude, longitude, altitude).
- **Stat tiles** — total races hosted, first and last year of use.
- **Race history table** — all races held at the circuit listed in reverse chronological order, showing year, race name, race winner, and winning constructor. Data is fetched with a single SPARQL query using `OPTIONAL` to gracefully handle historic races with missing result data.
- **Fastest laps** — the five all-time fastest individual laps recorded at the circuit, sourced from the lap times data (filtered to lap duration $> 60{,}000$ ms to exclude incomplete laps) and ordered by ascending milliseconds.

4.1.5. Races Section

The races list shows a featured card for the most recent race with full podium display, followed by a filterable list of all races. The race detail page features:

- Hero section with circuit, country, date, and round.
- Stat tiles: starters, race laps, fastest lap time and driver.
- Podium display (top 3 finishers).
- Full results table with grid position, laps completed, race time/gap, **finish status** (e.g. "+1 Lap", "Engine", "Collision"), and points.
- **Pit stop section** showing per-driver stop count, total pit time, and fastest individual stop.
- Qualifying results with Q1/Q2/Q3 times.

4.1.6. Seasons Section

The seasons list page renders one card per season sorted most-recent first. Each card shows the year prominently, race count, driver count, team count, and the driver with the most race wins that year. Client-side search and sort (by year or race count) operate over the rendered cards and a mirrored hidden table without any server round-trips. The view defaults to cards rather than table, reflecting the browsing-first use case for seasons.

The season detail page is the richest summary page in the application:

- **Race calendar table** — every round of the season with round number, race name, circuit, country, date, and race winner with their constructor. The winner is derived in SPARQL using `OPTIONAL { ?res f1:positionOrder 1 }` so rounds with no result data (e.g. future races) are still listed.
- **Driver championship standings** — final standings showing position, driver name, total points, and wins. Because the `driver_standings` table stores cumulative snapshots after every round, the query orders by round descending and Python retains only the first (latest) row per driver, yielding the end-of-season standings efficiently.
- **Constructor championship standings** — same approach applied to `constructor_standings`, showing team, points, and wins.

4.1.7. LLM Assistant

The public interface now includes an `/assistant/` page where users can ask natural-language questions about the Formula 1 knowledge graph. The server uses a Gemini model with a hardcoded schema/context prompt describing the main classes, predicates, and identifier conventions of the RDF graph. The workflow is two-stage:

- Gemini first generates a read-only `SELECT` SPARQL query.
- The application executes that query against GraphDB and sends the resulting bindings back to Gemini, which returns a natural-language answer.

The generated SPARQL is never shown to the end user. For safety, only `SELECT` queries are accepted; any attempt by the model to generate an update query is rejected server-side.

4.2. Admin Panel

The admin panel is accessible at `/admin-panel/` and requires login with a Django staff account. It features:

- **Login page** — standalone page with the site aesthetic, staff-only.
- **Dashboard** — live entity counts with quick-add shortcuts.
- **Entity management** for Drivers, Constructors, Circuits, Races, and Seasons: list view with inline Edit / Delete actions and Add form.
- **Relation-aware editing** — entity forms can modify RDF object properties such as the team/constructor linked to a driver.
- **Race results CSV import** — upload, validate, preview, and confirm batch result imports for a selected race.
- **Data quality checks** — dedicated page highlighting races without circuits, drivers without constructors, duplicate IDs/labels, dangling references, and broken result links.
- **Rollback of batch operations** — the most recent recorded batch import can be reversed from the admin interface.
- **Confirm-delete modal** — clicking Delete opens a modal requiring confirmation before the SPARQL DELETE is executed.
- **Sidebar navigation** — sticky left nav with active state highlighting the current section.
- **Flash messages** — success and error messages displayed after each operation.

4.3. Design System

The UI is built on a set of CSS custom properties (`-red`, `-dark`, `-panel`, `-grey`, etc.) that support both dark and light themes. The theme is persisted in `localStorage` and applied before paint to avoid a flash of unstyled content. Typography uses *Barlow Condensed* (headings) and *Barlow* (body text) from Google Fonts. The colour palette mirrors the official F1 brand: red (`#E10600`) on a near-black background (`#15151E`).

5. Conclusions

This project successfully demonstrates the end-to-end lifecycle of a semantic web application: from raw relational CSV data to an RDF knowledge graph, through SPARQL queries, to a fully functional web interface.

5.1. Key Achievements

- **Full RDF pipeline**: A Python script converts 13 CSV files into a consistent N-Triples RDF graph with correct datatype annotations, object property links between entities, and `rdfs:label` triples for the main browsable entities.
- **Efficient SPARQL queries**: The anchoring strategy (using unique property predicates rather than `rdf:type`) proved essential for acceptable query performance over a graph of millions of triples. Queries that naively used `rdf:type` were orders of magnitude slower.
- **Derived data without denormalisation**: Metrics such as wins, podiums, and championship points are computed at query time using SPARQL aggregation functions (`COUNT`, `SUM(IF(...))`, `MIN`, `MAX`). This keeps the graph normalised and ensures consistency.
- **Full CRUD via SPARQL UPDATE**: The admin interface demonstrates that GraphDB can serve as the single source of truth for both reads and writes, with no relational database involved for domain data.
- **Operations-oriented admin tooling**: Beyond CRUD, the admin now supports relation editing, batch race-result imports with dry-run previews, rollback of recorded import operations, and graph integrity checks.
- **Natural-language query interface**: A Gemini-backed assistant translates user questions into safe read-only SPARQL, executes them against GraphDB, and returns human-readable answers.
- **Aesthetic and usability**: The application provides a polished consumer-grade interface with dark/light theme switching, responsive layout, card/table view toggles, client-side search and sort, and accessible navigation.

5.2. Challenges and Lessons Learned

- **xsd:gYear comparison**: Race years stored as `xsd:gYear` cannot be compared directly with integer literals; they must be coerced to strings using `STR()`.
- **Sprint results contamination**: Sprint race results shared the `f1:resultId` anchor and `f1:race` link with regular results, causing duplicate rows. The issue was resolved by filtering on the URI pattern of the result entity, although the sprints data was excluded from the graph entirely to prevent future issues.
- **Status join ambiguity**: Joining via `f1:statusId` required care because result entities and status entities both carry this property; an additional `FILTER` on the status entity URI was needed to prevent cross-joins.

- **GROUP BY and SAMPLE()**: Season calendar queries must return one winner row per race when winners are represented as separate result entities. `SAMPLE()` allowed grouping while retaining an arbitrary winner value without requiring a subquery.

5.3. Future Work

- Extend the batch import pattern beyond race results to qualifying, pit stops, lap times, and standings.
- Implement a public SPARQL explorer endpoint, allowing power users to run custom queries against the knowledge graph.
- Integrate live data from the current season via the Ergast API.
- Replace the hardcoded assistant schema prompt with a richer ontology-aware context builder generated from the graph itself.

6. Configuration Required to Run the Application

6.1. Prerequisites

- **Python 3.11+** with `pip`
- **GraphDB 10** (free edition) running locally or remotely

6.2. Step 1 — Clone and Install Dependencies

```
# From the project root:
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

6.3. Step 2 — Configure Environment Variables

Create a `.env` file in the project root:

```
DJANGO_SECRET_KEY=your-secret-key-here
DJANGO_DEBUG=True
GRAPHDB_BASE_URL=http://localhost:7200
GRAPHDB_REPOSITORY=ws-formula1
GEMINI_KEY=your-gemini-api-key
# Optional: override the default Gemini model
# GEMINI_MODEL=gemini-2.5-flash
# Optional: if GraphDB requires authentication
# GRAPHDB_USERNAME=admin
# GRAPHDB_PASSWORD=your-password
```

6.4. Step 3 — Prepare the Knowledge Graph

1. Place the Ergast CSV files in `data/raw/`.
2. Run the conversion script to generate N-Triples:

```
python scripts/csv_to_rdf.py
# Output: data/rdf/f1_dataset.nt
```

3. Open GraphDB Workbench at `http://localhost:7200`.
4. Create a repository named `ws-formula1` with **Ruleset: RDFS+ (Optimized)**.
5. Go to **Import** → **Upload RDF files**, select `data/rdf/f1_dataset.nt`, and import into the named graph `http://example.org/graph/formula1`.

6.5. Step 4 — Initialise Django and Create Admin User

```
python manage.py migrate
python manage.py createsuperuser
# Follow the prompts to set username and password
```

6.6. Step 5 — Run the Development Server

```
python manage.py runserver
# Application available at http://127.0.0.1:8000
# Admin panel at http://127.0.0.1:8000/admin-panel/
```

6.7. Summary of URLs

URL	Description
`/`	Homepage with live stats
`/assistant/`	Gemini-backed natural-language assistant
`/drivers/`	Drivers list
`/constructors/`	Constructors list
`/circuits/`	Circuits list
`/races/`	Races list
`/seasons/`	Seasons list
`/admin-panel/`	Admin dashboard (login required)
`/admin-panel/login/`	Admin login page
`/admin-panel/imports/race-results/`	Race results CSV import
`/admin-panel/data-quality/`	Data quality checks